

Contents

1. Introductory Problems	3
1.1. Concepts	3
1.1.1. Basic c++ syntax	3
1.1.1.1. Question	3
1.1.1.2. Data Types	4
1.1.1.3. Variables	4
1.1.1.4. Input/Output	5
1.1.1.5. Conditional Statements	5
1.1.1.6. Loops	5
1.1.1.7. Classes/Structs	5
1.1.1.8. Arrays/Vectors	5
1.1.1.9. Functions	5
1.1.2. Time Complexity	5
1.1.3. Pointers	5
1.1.4. Vectors in Depth	7
1.1.5. Recursion	7
1.1.6. Sorting	8
1.1.7. Binary Search	8
1.1.8. Lower Bound and Upper Bound	10
1.1.8.1. lower_bound() & upper_bound() with custom sorting.	12
1.1.9. Permutations	12
1.1.9.1. next_permutation()	13
1.1.10. Backtracking	14
1.2. CSES Practice Question	16
1.2.1. Weird Algorithm	16
1.2.2. Missing Number	18
1.2.3. Repetitions	19
1.2.4. Increasing Array	20
1.2.5. Permutations	22
1.2.6. Number Spiral	24
1.2.7. Two Knights	26
1.2.8. Two Sets	28
1.2.9. Bit Strings	30
1.2.10. Trailing Zeros	31
1.2.11. Coin Piles	32
1.2.12. Palindrome Reorder	33
1.2.13. Gray Code	35
1.2.13.1. Solution 1	35
1.2.13.2. Solution 2	36
1.2.14. Tower of Hanoi	38
1.2.15. Creating Strings	40
1.2.16. Apple Division	41
1.2.17. Chessboard and Queens	43
1.2.18. Raab Game I	45
1.2.19. Mex Grid Construction	47
1.2.20. Knight Moves Grid	49

1.2.21. Grid Coloring I	52
1.2.22. Digit Queries	54
1.2.23. String Reorder	56
1.2.24. Grid Path Description	59

1. Introductory Problems

1.1. Concepts

1.1.1. Basic c++ syntax

1.1.1.1. Question

Accept the number of students from user. Accept their names and marks. Print the Name(s) of students who scored the highest percentage. We deliberately use C++ features useful for programming contests.

Solution:

```
1
2  #include <bits/stdc++.h>
3  using namespace std;
4
5  double calcPercent(int numerator, int denominator){
6      return numerator * 100.0 / denominator;
7      // An example of a single line comment
8      /* An example of a multiline comment
9          numerator / denominator * 100.0 will first do integer division.
10         That's why we multiply by 100.0 first and then divide by the
11         denominator. */
12 }
13 struct Student{
14     string name;
15     pair<int, int> marks;
16     double percent;
17     Student(); // this is a default constructor
18
19     // this is a parameterized constructor
20     Student(string name, pair<int, int> marks) {
21         this->name = name;
22         this->marks = marks;
23         percent = calcPercent(marks.first, marks.second);
24     }
25 };
26
27 // Program execution begins from here
28 int main() {
29     int n;
30     cin >> n;
31
32     // to create an array of n Students the default constructor was necessary
33     Student arr[n];
```

```

34  double maxPercentage = 0.0;
35
36  for(int i = 0; i < n; i++){
37      string name;
38      pair<int, int> marks;
39      cin >> name >> marks.first >> marks.second;
40
41      // calling parameterized constructor
42      arr[i] = Student(name, marks);
43      maxPercentage = max(arr[i].percent, maxPercentage);
44  }
45
46  // a vector is a resizable array with some useful functions
47  // memory is automatically allocated in a vector
48  vector<Student> best;
49
50  for(int i = 0; i < n; i++)
51      if(arr[i].percent == maxPercentage) {
52          // push_back() adds the student to the end of the vector
53          best.push_back(arr[i]);
54      }
55
56  cout << "Names, Marks and Percentages of top scorers!" << endl;
57  for(int i = 0; i < best.size(); i++) {
58      cout << "Name: " << best[i].name;
59      cout << ", Marks: " << best[i].marks.first << "/" <<
        best[i].marks.second;
60      cout << ", Percentage: " << best[i].percent << endl;
61  }
62
63  return 0; // end code
64 }

```

While this isn't the only way to solve the question, the code should cover the most basic c++ syntax.

1.1.1.2. Data Types

This code contained the data types `int` (Integer which is a non decimal number) , `double` (Decimal Number), `string` (Text) and `pair<int,int>`. A `pair` is a datatype that can be a combination of 2 other data types and each individual part can be accessed with `.first` and `.second`. In this case it was 2 `int`'s but it could be a pair of `int` and `string` and much more.

1.1.1.3. Variables

Variables are strongly typed in c++ which means you must specify their datatype and then their name.

1.1.1.4. Input/Output

Input and output is done with `cin` and `cout` and angle brackets `>>` for input and `<<` for output.

1.1.1.5. Conditional Statements

Conditional Statements are represented with `if`. The part inside the `if` block runs if the condition is true. You can also use `else` which triggers if the `if` block above is `false` and create if else ladders with `else if` which triggers if the above `if` and `else if` blocks were `false`.

1.1.1.6. Loops

A loop in the example is a `for` loop, which has 3 parts, the first part initializes a variable. The second part is the condition to determine if the loop should continue and the 3 part is what happens at the end of the loop block which is usually to update the variable initialized in the first part.

1.1.1.7. Classes/Structs

In this program we made a `struct` because it's easier to use than a `class`. They work in nearly the same way though and the only difference really is that members in a `struct` are `public` by default but members in a `class` are `private` by default.

1.1.1.8. Arrays/Vectors

An array is a list of many of the same datatype. In this program we made an array of `Students` which is our own datatype. We also made a vector, which unlike an array, has a dynamic size.

1.1.1.9. Functions

A function is something that accepts parameters and returns a value. This includes our `calcPercent` function and the 2 constructors used to make `Student`.

More about `c++` syntax can be learned [here](#).

1.1.2. Time Complexity

Time Complexity is simply a measure of how much longer it takes a program to run as the input size grows larger. We represent it by using something called Big-O Notation. For instance, say we have a program that is $O(n)$, this means that the function is linear, i.e. if you double the input size, the program will take twice as long. A program with time complexity $O(n^2)$ will take 4 times as long for twice the input size.

Whenever you are solving a question, always calculate the time complexity of your algorithm. When you plug in the maximum input sizes into your time complexity, the amount of time it should take should be less than 10^8 because that's usually how many operations occur in one second.

1.1.3. Pointers

Unlike in other higher level programming languages which you may be familiar with, `c++` allows you to have full control over how to allocate memory. This is achieved by using `pointers`.

A pointer is a variable that stores a memory location instead of the value. Here's an example of a code which uses pointers and we'll explain what it does:-

```

1
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main(){
6     int a = 5; //made an int variable with value 5.
7     int *b = new int(7); //made an integer pointer with value 7 at that memory
    location.
8     int &c = a; //made an int variable which refers to the same memory location
    as a.
9     int *d = &a; //made an int pointer which points to the same memory location
    as a.
10
11     cout << a << " " << *b << " " << c << " " << *d << endl;
12     c = 9; //also changes a and d
13     cout << a << endl; //9
14     *d = 15; // also changes a and c
15     cout << a << endl; //15
16
17     delete b; //every time you use "new" you must always "delete" the pointer to
    prevent memory leaks
18     return 0;
19 }

```

While we have written comments, we'll still go deeper to explain the most important lines:

- `int a = 5` creates a variable `a` which has a value 5.
- `int *b = new int(7)` makes a pointer `b`, which at its memory location has the value 7.
- `int &c = a` makes a variable `c` which has the same value that `a` has. This means that modifying one of them will modify the other. They are the same value with 2 different names.
- `int *d = &a` makes a pointer `d` which stores the memory location of `a`. This also makes `d` the same as `a` and `c` however `d` is a memory location which at the location has the same value as `a` and `c`.
- `cout << a << " " << *b << " " << c << " " << *d << endl` outputs `a`, `*b` which is the value at memory location `b`, `c` and `*d` which is the value at memory location `d`.
- `c = 9` changes the values of `a` and the value at memory location `d` to 9.
- `*d = 15` changes the value at memory location `d` to 15 which also changes `a` and `c`.
- `delete b` is the most important line. **Every time you use the keyword `new`, you must use `delete` to free up the memory.** Otherwise, that memory will remain allocated to nobody after your program has ended. This is called a memory leak and the only way to free up such "leaked" memory is by restarting your computer.

To summarize the new syntax of pointers:

1. `int *x` creates a pointer which stores a memory location.
2. `*x` **dereferences** the pointer allowed you to see the value
3. `int &x` allows you to pass another variable by reference, i.e. Both variables share the same memory location.

4. `&x` gives the memory location of the variable `x`.

1.1.4. Vectors in Depth

We're going to go into **vectors** in a little more depth. As stated before **vectors** are almost the same as arrays except they are dynamic, meaning the elements can be added and removed but only at the end. This is done by the `push_back()` and `pop_back()` functions.

The way **vectors** make this efficient time wise without wastes a lot of memory is by allocating some memory x in a row. When you `push_back()` an element such that it now exceeds x , it moves the entire allocated memory to a new location and allocates memory worth $2x$. This means that the time complexity of inserting elements into a **vector** is close, but not quite $O(1)$. This is called amortized $O(1)$ because it looks at the average instead of each single operation and because **vector** resizes occur infrequently.

Note that **vectors** constant factors are bigger than arrays, which means for questions where every little efficiency matters to solve the question, if you don't need a **vector**, don't use one. However in every other case, it's much safer and more convenient to use **vectors** instead of arrays. The main reason being that:-

1. It's easier to initialize all values in a vector

```
vector<int> v(5,-1)//Initializes vector of size 5 filled with -1
```

2. When passing an array to a function, it **always** passes by reference. Passing by reference simply means that the function can make changes to the original array. Sometimes however we wish to pass by value, meaning that a new copy is made. With vectors we have such freedom to choose.

More technical details about **vectors** can be found [here](#).

1.1.5. Recursion

Recursion is the concept of calling some function inside of itself. Say we want to compute the factorial of a number n . We can do:

```
1
2  #include <bits/stdc++.h>
3  using namespace std;
4
5  int fact(int n){
6      if(n == 1) //base case
7          return 1;
8      return n * fact(n - 1); //recursion
9  }
10
11 int main(){
12     int n;
13     cin >> n;
14     cout << fact(n) << endl;
15
16     return 0;
17 }
```

C++

Every recursive algorithm has 2 main things:

1. A base case. Some failure point at which you must return a known value. In this code it was $n = 1$ and we returned 1 for that base case. You can always have multiple base cases if necessary.
2. Recursion. This is the part where you call the original function on a smaller problem than the original. In this case we call `fact(n-1)` and then multiply it by `n` to get `fact(n)`.

Fun fact: It's proven that any recursion function can be written with a loop! Loops are more efficient than recursion, so if it is easier to write a loop you should. However, some programs are too hard to convert to loops so you should stick to recursion.

1.1.6. Sorting

To sort a data structure like an array or vector, c++ has its own sort function for this:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main(){
5
6      int arr[] = {3,4,6,2,5,1};
7      vector<int> v = {6,2,4,5,1,3};
8
9      sort(arr, arr+6); //Sorts the array {1,2,3,4,5,6}
10     sort(v.begin(), v.end()); //Sorts the vector {1,2,3,4,5,6}
11
12     return 0;
13 }
```

As you can see, the sort function accepts 2 pointers, the start position of the sort and one position after the end of where you want the elements sorted. `arr` is a pointer to the start of the array. You can add a number to this pointer to jump ahead that many places. `arr + 6` is one position past the end of the array because we want to sort the entire array in this case although you don't always have to. `v.begin()` is a pointer to the start of the vector and `v.end()` points one place after the last element of the vector. You can also add a value to `v.begin()` to jump to other positions in the vector to sort only a part of it.

The time complexity of `std::sort` is $O(n \log n)$.

1.1.7. Binary Search

Let's say you want to find a certain number in a list of numbers to see if it exists. Normally the way you would do this is by iterating over each element in the array and checking if it matches the element you're looking for. The time complexity of this is $O(n)$. However, if we were to first sort the array, we can find a number in $O(\log n)$!

You may not have realized it, but you have probably already used binary search in your life at least once! When ever you use a dictionary, you don't search word by word to see if it matches the word you are looking for, you instead apply something similar to binary search. Say you're looking for the word "computers", you find open to the middle of the dictionary. You'll probably be in the m-n section which is too far ahead, so you jump back half way.

You repeat this until you get to the c section. However, you may be in the ca section which is now behind, so you jump forward by half way forward until you reach “computers”. While you may not be doing exactly this, we can use this method to find things really quickly.

The main steps are as follows:

1. Starting in the middle
2. If you are equal to the target you have found the value! Otherwise, go to 3.
3. If you are less than the target, eliminate the left and then jump to the middle of the right half then go back up to 2. If not, go to 4
4. If you are more than the target, eliminate the right half and jump to the middle of the left, then go to 2.

Let’s see the algorithm in action:-

Let’s say we have the following sorted array:

{1, 4, 4, 5, 6, 6, 7, 9, 13, 15, 16, 18, 21, 30}

And let that the target number we are looking for be 18. Let the be the variables `left` = 0, `right` = 13, and `middle` = $\frac{\text{left} + \text{right}}{2} = \frac{0+13}{2} = 6$ which is the average of `left` and `right`.

{1, 4, 4, 5, 6, 6, 7, 9, 13, 15, 16, 18, 21, 30}

Now we can compare the value of `middle` with our target 18. As you can see, `middle` < 18. This tells us that our target value lies to the right of `middle`. We can now update `middle` by first making `left` = `middle` + 1 = 6 + 1 = 7, then make `middle` = $\frac{\text{left} + \text{right}}{2} = \frac{7+13}{2} = 10$.

{1, 4, 4, 5, 6, 6, 7, 9, 13, 15, 16, 18, 21, 30}

Once again we are too low, so we set `left` = `middle` + 1 = 10 + 1 = 11, and then `middle` = $\frac{\text{left} + \text{right}}{2} = \frac{11+13}{2} = 12$.

{1, 4, 4, 5, 6, 6, 7, 9, 13, 15, 16, 18, 21, 30}

This time we’re too high, so now we set `right` = `middle` - 1 = 12 - 1 = 11 and then `middle` = $\frac{\text{left} + \text{right}}{2} = \frac{11+11}{2} = 11$

{1, 4, 4, 5, 6, 6, 7, 9, 13, 15, 16, 18, 21, 30}

Now `middle` is equal to 18 our target! And it only took us 4 steps. If we had iterated normally it would’ve taken 12.

Here the implementation of this, where the user will supply us a sorted list of numbers and a target value for us to find. We output whether the value exists and then it’s position in the list:

```
1
2  #include <bits/stdc++.h>
3  using namespace std;
4
5  int main(){
6      ios_base::sync_with_stdio(0);
7      cin.tie(0);
```

C++

```

8     cout.tie(0);
9
10    int n, t;
11    cin >> n;
12    vector<int> v(n);
13    for(int i = 0; i < n; i++)
14        cin >> v[i];
15
16    int l = 0, r = n - 1, m ;
17    while(l <= r){
18        m = (l + r)/2;
19        if(v[m] == t){//if the number at m meets the target
20            cout << "YES" << endl;
21            cout << m << endl;
22            return 0;
23        }
24        else if(v[m] < t)
25            l = m + 1;//eliminate the left(lesser) half
26        else if(v[m] > t)
27            r = m - 1;//eliminate the right(greater) half
28    }
29    cout << "NO" << endl;
30    return 0;
31 }

```

1.1.8. Lower Bound and Upper Bound

Usually whenever we do binary search, we rarely ever want to know if a value is actually there or not, rather we'd like to know 2 things:-

1. The first number in the list greater than or equal to the number. This is called finding the **lower bound**.
2. The first number in the list **strictly** greater than the number. This is called finding the **upper bound**.

To be able to compute the **lower bound** and **upper bound** of some number t , we only need to modify the while loop of our earlier binary search algorithm:

Lower Bound:

```

1    int l = 0, r = n - 1, m, lb = -1;
2    while(l <= r){
3        m = (l + r)/2;
4        if(v[m] < t)
5            l = m + 1;
6        else if(v[m] >= t){// >= instead of > because it lower bound.
7            lb = m;//we set the lower bound to the middle

```

C++

```

8      r = m - 1; // and then eliminate the right half.
9  }
10 }
11
12 cout << lb << endl;

```

Upper Bound:

```

1  int l = 0, r = n - 1, m, ub = -1;
2  while(l <= r){
3      m = (l + r)/2;
4      if(v[m] <= t) // < to <= the equal condition isn't missed.
5          l = m + 1;
6      else if(v[m] > t){
7          ub = m; // we set the upper bound to the middle
8          r = m - 1; // and then eliminate the right half.
9      }
10 }
11
12 cout << ub << endl;

```

You can try the algorithm of lower bound and upper bound on an array and with a target value and see how this works.

Now lucky for you, c++ comes with its own upper bound and lower bound functions! Here's their use cases:

Note: `upper_bound()` and `lower_bound()` only work properly on sorted lists in ascending order. They will output the wrong value otherwise

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main(){
5      ios_base::sync_with_stdio(0);
6      cin.tie(0);
7      cout.tie(0);
8
9      int n, t;
10     cin >> n;
11     vector<int> v(n);
12     for(int i = 0; i < n; i++)
13         cin >> v[i];
14
15     vector<int>::iterator lb = lower_bound(v.begin(), v.end(), t); // lower_bound()
    returns an iterator to the position of the lower bound of t.

```

```

16  vector<int>::iterator ub = upper_bound(v.begin(),v.end(),t);//upper_bound()
    returns an iterator to the position of the upper bound of t.
17
18  cout << lb - v.begin() << endl;//difference between lb and v.begin() tell
    you the index of the lower bound.
19  cout << ub - v.begin() << endl;//difference between ub and v.begin() tell
    you the index of the upper bound.
20  return 0;
21 }

```

As you can see from the code above:-

- `lower(v.begin(),v.end(),t)` returns an iterator to the lower bound of `t`.
- `upper(v.begin(),v.end(),t)` returns an iterator to the upper bound of `t`.

To get the index, we simply do `lb - v.begin()` and `ub - v.begin()` because that takes the difference in memory location.

You now might wonder, how do you get the largest element lesser than or the largest element less than or equal to. This can be achieved by subtracting 1 to the lower bound and upper bound respectively.

1.1.8.1. `lower_bound()` & `upper_bound()` with custom sorting.

Sometimes your vector may not be sorted in ascending order. Sometimes it might be descending, sometimes it could be some custom ordering. In these cases it's important to understand what `lower_bound()` and `upper_bound()` are actually doing.

`lower_bound(first, last, val, comp())` returns an iterator of the first value where `comp(*it,val)` is false

`upper_bound(first, last, val, comp())` returns an iterator of the first value where `comp(val,*it)` is true

By default, the `comp()` function is `operator<()`, however this can be changed to `greater<int>()` which returns true if the first number is more than the second number, which is needed for it to work properly on a descending list. Note however that `upper_bound()` and `lower_bound()` may not actually give the mathematical definition of lower bound and upper bound if you use it on a descending list. Apply a correction factor as needed.

1.1.9. Permutations

Let's say you are given a string, and you wish to list out all possible permutations of the string. For instance "abcde". You could probably write out all $5! = 120$ possibilities on your own but what rule could you do to make a computer do it? Try listing the permutations yourself and see if you come up with sometime before reading onwards.

Alright, here's the method:

Let's first list out the permutations of a string of length 3 "abc":

1. "abc"
2. "acb"
3. "bac"
4. "bca"

5. "cab"
6. "cba"

As you can see, we went through all permutations starting from the string sorted in ascending order and then in descending order. To then go from one permutation to the next, there are 3 steps:

1. Scan from right to left. The first position where you find the current element less than the next one ($\text{str}[i] < \text{str}[i+1]$). This position is the pivot.
2. Swap the element at the pivot with its upper bound to the left of it (See Section 1.1.8)
3. Reverse all elements after the pivot.

Try this out with your letter sequence and you'll see that this is probably what you do intuitively without realizing it.

Here's the code for that algorithm:

```
1
2  #include <bits/stdc++.h>
3  using namespace std;
4
5
6  int main(){
7
8      string str = "abcd";
9
10     do {
11         cout << str << endl;
12     } while(next_permutation(str.begin(),str.end()));
13
14     return 0;
15 }
```

1.1.9.1. next_permutation()

Fortunately for you, c++ already has a function for you that generates the next permutation! Here's the same code we wrote above but using `next_permutation()`:

```
1
2  #include <bits/stdc++.h>
3  using namespace std;
4
5  bool permute(string &str){
6      for(int i = str.length() - 2; i >= 0; i--){
7          if(str[i] < str[i + 1]){//pivot finding
8
9              int ub_idx = lower_bound(str.begin() + (i + 1), str.end(), str[i],
              greater<int>()) - 1 - str.begin();//finds the upper bound of element at
              pivot.
          }
```

```

10     swap(str[i], str[ub_idx]); //swaps the ub and the element at the pivot
11     reverse(str.begin() + (i + 1), str.end()); //reverses the elements after
    the pivot
12
13     return true; //succesfully produced the next permutation
14
15 }
16 }
17
18 return false; //failed to produce the next permutation. This happens when
    the string is in descending order because that the last permuatation.
19 }
20
21 int main(){
22
23     string str = "abcd";
24
25     do {
26         cout << str << endl;
27     } while(permute(str));
28
29     return 0;
30 }

```

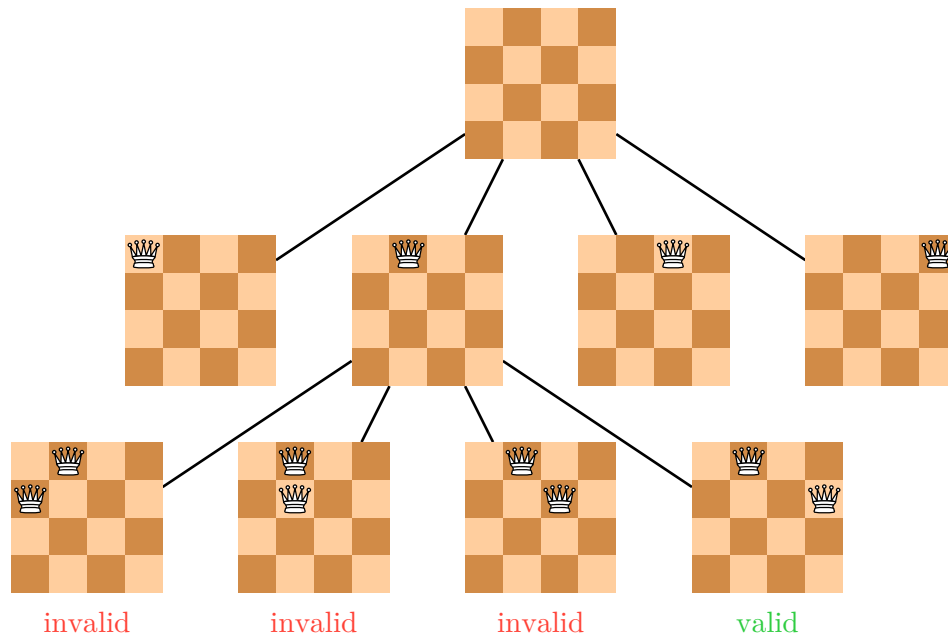
1.1.10. Backtracking

A backtracking algorithm is one where you recursively go the all possibilities and then backtrack at invalid solutions. Let's use an example to explain this better.

Say we want to know for a $n \times n$ chess board, how may ways are there to place n queens, such that two queens never attack each other.¹

This problem can be solved by using backtracking. We can start by placing the first queen in all positions on the first row, for each of those position, see which positions you can place a queen in the second row and so on. Let's looks at some partial solutions when $n = 4$.

¹If you don't know anything about chess, 2 queens attack each other if they both lie on the same row, column, or diagonal.



As you can see, we start with an empty board, then we place a queen in all positions on the first row. Then we place the next queen on the next row in a valid position and then go from there.

To write the code for this. We need 4 arrays, one for every row, columns, and both diagonals. If `row[i]` is true, that means there's a queen in that row and we can't place a queen there. The indexes of the two diagonals will be as follows:

0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

First diagonal

3	2	1	0
4	3	2	1
5	4	3	2
6	5	4	3

Second diagonal

If a queen is on row i and column j , then it will be on `row[i]`, `column[j]`, `diag1[i + j]` and on `diag[(n-1) - j + i]`. Then for the next row, a queen can't be placed on this row, columns, and diagonal.

Here's the code of the implementation for this algorithm:

```

1
2  #include <bits/stdc++.h>
3  using namespace std;
4
5  int n, ans = 0;
6  vector<bool> col, diag1, diag2;
7
8  void findPositions(int i = 0){
9      if(i == n){//If true, we successfully placed all the queens in an
10         arrangement.
11         ans++;
12     }
13 }
```

C++

```

11     return;
12 }
13
14 for(int j = 0; j < n; j++){
15     if(col[j] || diag1[i+j] || diag2[(n-1)-j+i]) //The new queen would be
        attacked
16         continue;
17     col[j] = diag1[i+j] = diag2[(n-1)-j+i] = true; //Placing the queen on the
        current spot
18     findPositions(i+1);
19     col[j] = diag1[i+j] = diag2[(n-1)-j+i] = false; //Removing queen for the
        current spot
20 }
21 }
22 int main(){
23     ios_base::sync_with_stdio(0);
24     cin.tie(0);
25     cout.tie(0);
26
27     cin >> n;
28     col.resize(n);
29     diag1.resize(2*n-1);
30     diag2.resize(2*n-1);
31     findPositions();
32
33     cout << ans << endl;
34     return 0;
35 }

```

The `resize()` function of a vector is used when you wish to specify the size of a **vector** after its initialization. The `findPositions()` function has a default value of `i = 0`, so if the parameter isn't supplied it assumes the value to be `0`. Also observe that we didn't use a **row vector**, because the backtracking algorithm ensures that we are placing the new queen on a new row each time.

The complexity of this code is $O(n!)$ which grows very quickly. Solving the question for high values of n takes a very long time. The highest anybody has computed is $q(27) = 234907967154122528$ and this took over a year of computing! ([See here](#)).

1.2. CSES Practice Question

1.2.1. Weird Algorithm

[Question - Weird Algorithm](#) [Backup Link](#)

Explanation :

1. Modulus Function (%): Used to check the parity of n
 - If $(n \% 2 == 0)$, n is even, so divide n by 2
 - If $(n \% 2 == 1)$, n is odd, so multiply n by 3 and add 1
2. While Loop: Continues the process until n becomes 1.
 - The loop runs as long as n is not 1, applying the above rules in each iteration.
 - Print each value of n to track the sequence.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      long long n; // Use long long to handle large values of n
6      cin >> n;
7      cout << n << " "; // Print the starting number
8
9      // Continue until n becomes 1
10     while (n != 1) {
11         // Case 1: n is even → halve it
12         if (n % 2 == 0) n /= 2;
13
14         // Case 2: n is odd → apply 3n + 1
15         else n = 3 * n + 1;
16
17         // Print current value after operation
18         cout << n << " ";
19     }
20
21     return 0;
22 }
```

C++

1.2.2. Missing Number

[Question - Missing Number](#) [Backup Link](#)

Explanation :

We use a simple mathematical trick: calculate the expected sum of numbers from 1 to n using the arithmetic progression formula

$$\frac{n(n+1)}{2}$$

Then subtract the actual sum of the given numbers to reveal the missing number, as this difference represents the value that's absent, elegantly avoiding any searching or sorting in a fast way.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      long long n, x, total = 0;
6      cin >> n;
7
8      for (int i = 0; i < n - 1; i++) {
9          cin >> x;
10         total += x;
11     } // Read n-1 numbers and sum them
12
13     long long sum = n * (n + 1) / 2; // Expected sum of 1 to n
14
15     // The missing number is the difference
16     cout << sum - total << endl;
17     return 0;
18 }
```

C++

1.2.3. Repetitions

[Question - Repetitions](#) [Backup Link](#)

Explanation : This program finds the longest stretch of the same character in a string.

It goes through each character one by one:

1. If it's the same as the previous one, it extends the current streak.
2. If it's different, it resets the count.
3. It keeps track of the maximum streak found.

Finally, it prints the length of that longest consecutive sequence.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      string s;
6      cin >> s;
7      int maxLen = 1, current = 1;
8
9      for (int i = 1; i < s.length(); i++) {
10         // Check if current character matches the previous one
11
12         // Increment current length of consecutive characters
13         if (s[i] == s[i - 1]) current++;
14
15         // Reset current to 1 if characters differ
16         else current = 1;
17
18         // Update maxLen if current is larger
19         maxLen = max(maxLen, current);
20     }
21
22     cout << maxLen << "\n";
23     return 0;
24 }
```

C++

1.2.4. Increasing Array

[Question - Increasing Array](#) [Backup Link](#)

Explanation :

We need to make the given array non-decreasing — that is, every element must be at least as large as the one before it. Whenever a number is smaller than the previous one, we must increase it until the condition $a[i] \geq a[i-1]$ holds. The problem asks for the total number of increments required to achieve this.

Algorithm (Step by Step Flow):

1. Read the first element and store it as prev.
2. Iterate through the rest of the array:
3. If $\text{current} \geq \text{prev}$, move on — the order is fine.
4. If $\text{current} < \text{prev}$, we need to increase it by $(\text{prev} - \text{current})$.
5. Add this difference to the total count and update $\text{current} = \text{prev}$.
6. Continue until all elements are processed.
7. Output the total count of increments. required.

Code :

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n, prev;
6      long long operations = 0; // total number of increments needed
7      cin >> n >> prev;
8
9      // process the rest of the array
10     for (int i = 1; i < n; ++i) {
11         int current;
12         cin >> current; // read the current element
13
14         // if the current element is smaller than the previous,
15         // we need to increment it to match 'prev' (to keep array non-
16         // decreasing)
17         if (current < prev) {
18             // count how many increments are required
19             operations += prev - current;
20             // simulate the increment (virtually update current)
21             current = prev;
22     }
```

C++

```
23     prev = current; // update 'prev' for the next iteration
24 }
25
26 cout << operations << '\n';
27 return 0;
28 }
```

1.2.5. Permutations

[Question - Traffic Lights](#) [Backup Link](#)

Explanation :

The trick we exploit here is to first print all the numbers up to n of one parity (odd or even), and then print all the numbers of the opposite parity. This is because the difference between consecutive odd numbers is always greater than 1.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      // Special case: if n = 2 or 3, it is impossible to arrange
9      // numbers from 1...n so that no two consecutive numbers
10     // differ by 1. Hence, print "NO SOLUTION".
11     if (n > 1 && n < 4)
12         cout << "NO SOLUTION";
13
14     // Base case: if n = 1, the only permutation is "1".
15     else if (n == 1)
16         cout << "1";
17
18     // Edge case: if n = 4, the general logic won't directly
19     // give a valid solution, so hardcode one valid answer.
20     else if (n == 4)
21         cout << "2 4 1 3";
22
23     // General case: n >= 5
24     else {
25         // First print all odd numbers in descending order
26         // This ensures that consecutive numbers differ by >= 2
27         for (int i = n; i >= 1; i -= 2)
28             cout << i << " ";
29
30         // Then print all even numbers in descending order
31         // This also ensures no two adjacent numbers differ by 1
32         for (int i = n - 1; i >= 1; i -= 2)
33             cout << i << " ";
34     }
```

C++

1.2.6. Number Spiral

[Question - Number Spiral](#) [Backup Link](#)

Intuitive Explanation :

The spiral fills outward in square layers, where layer L contains all cells with $\max(x, y) = L$. Each layer's diagonal cell (L, L) holds the value $L^2 - (L - 1)$ because at the L th layer, the value L^2 is at one of the edges and then to go from there to the diagonal, subtract $(L - 1)$. This value serves as our anchor point.

The key insight:

- Even layers fill downward then leftward, while odd layers fill rightward then upward. So for even layers, if you're on the rightmost edge ($x=L$), you subtract how far down you are from the diagonal; otherwise you're on the top edge, so add how far left you are.
- Odd layers work inversely: if you're on the top edge ($y=L$), subtract your leftward distance; otherwise you're on the left edge, so add your downward distance.

This directional pattern emerges because the spiral alternates its filling direction with each layer to maintain continuity.

Example: $y = 5, x = 3$

1	2	9	10	25
4	3	8	11	24
5	6	7	12	23
16	15	14	13	22
17	18	19	20	21

Algorithm (Step by Step Flow):

1. As $\max(5, 3) = 5$, It is on the 5th layer.
2. $L^2 - (L - 1) = 25 - (5 - 1) = 21$. 21 serves as our anchor point.
3. It is important to keep it mind that we are on an odd layer(as 5 is odd).
4. And as we have to go two cells to the left from our anchor point we subtract our leftward distance. Thus, $\text{answer} = 21 - 2 = 19$.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
```

C++


```

5     int t; // Number of test cases
6     cin >> t;
7     while (t--) {
8         // Cell coordinates
9         long long y, x;
10        cin >> y >> x;
11        long long layer = max(x, y); // Layer is max(x, y)
12        long long val = layer * layer - layer + 1; // Base value for layer's
        diagonal cell
13
14        if (layer % 2 == 0) // Even layer
15            if (x == layer) // Adjust for y
16                cout << val - (layer - y) << "\n";
17            else // Adjust for x
18                cout << val + (layer - x) << "\n";
19
20        else // Odd layer
21            if (y == layer) // Adjust for x
22                cout << val - (layer - x) << "\n";
23            else // Adjust for y
24                cout << val + (layer - y) << "\n";
25    }
26    return 0;
27 }

```

1.2.7. Two Knights

[Question - Two Knights](#)

[Backup Link](#)

Solution:

The problem asks: for each board size $k \times k$ (from $k = 1$ to $k = n$), count the number of ways to place two knights such that they do not attack each other.

Step 1: Count all possible placements

First, we count the total number of ways to place two knights on a $k \times k$ board without any restrictions. There are k^2 squares, and we need to choose 2 of them. This is simply:

$$\text{Total placements} = \binom{k^2}{2} = \frac{k^2(k^2 - 1)}{2}$$

Step 2: Subtract attacking pairs

Now we subtract the number of placements where the two knights attack each other. A knight attacks in an “L-shape”: it moves 2 squares in one direction and 1 square perpendicular to that.

The key insight is that **two knights that attack each other always fit inside a 2×3 or 3×2 rectangle**. Within each such rectangle, there are exactly **2 pairs** of squares that attack each other (the two diagonal corners of the “L”).

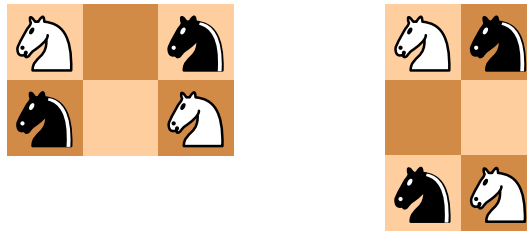


Figure 5: The 2 pairs of same coloured knights attack each other from the corners.

Step 3: Count the rectangles

- Number of 2×3 rectangles in a $k \times k$ board: $(k - 1) \times (k - 2)$
- Number of 3×2 rectangles in a $k \times k$ board: $(k - 2) \times (k - 1)$

Each rectangle contains 2 attacking pairs, so:

$$\text{Attacking pairs} = 2 \times (k - 1)(k - 2) + 2 \times (k - 2)(k - 1) = 4(k - 1)(k - 2)$$

Final Formula:

$$\text{Answer} = \frac{k^2(k^2 - 1)}{2} - 4(k - 1)(k - 2)$$

Examples:

- **k = 1:** Only 1 square, so 0 ways to place two knights. Answer = 0.
- **k = 2:** Total = $\binom{4}{2} = 6$ placements. No 2×3 or 3×2 rectangles fit, so 0 attacking pairs. Answer = 6.
- **k = 3:** Total = $\binom{9}{2} = 36$ placements. Attacking pairs = $4 \times 2 \times 1 = 8$. Answer = $36 - 8 = 28$.
- **k = 4:** Total = $\binom{16}{2} = 120$ placements. Attacking pairs = $4 \times 3 \times 2 = 24$. Answer = $120 - 24 = 96$.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      for (long long k = 1; k <= n; ++k) {
9          long long total = k * k;
10
11         // Total ways to place 2 knights on k*k board
12         long long ways = total * (total - 1) / 2;
13
14         // Subtract attacking pairs (only exist when k > 2)
15         if (k > 2)
16             ways -= 4 * (k - 1) * (k - 2);
17
18         cout << ways << "\n";
19     }
20
21     return 0;
22 }
```

C++

1.2.8. Two Sets

[Question - Two Sets](#) [Backup Link](#)

Intuitive Explanation :

The sum of the first n integers is given by : $\frac{n(n+1)}{2}$

Only when this sum is even can the set be split evenly, i.e. $n * (n - 1)$ should be divisible by four. Notice the values of n which satisfy the above condition are of the form $4n$ or $4n - 1$ (eg : 3, 4, 7, 8, 11, 12).

Every time the numbers can be paired off symmetrically: $\{n, n-3\}$ balancing $\{n-1, n-2\}$. Anything leftover (the magical trio 1,2,3) slots in with a final tidy arrangement. This pattern ensures the two sets always weigh the same, no matter how large n grows.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      // Check if the total sum n*(n+1)/2 is even (i.e., divisible by 2 after
      // halving),
9      // which requires n*(n+1) to be divisible by 4. If not, impossible to
      // split.
10     if (n*(n+1) % 4 != 0) cout<<"NO";
11
12     else {
13         cout<<"YES"<<endl;
14
15         // Vectors to store the two sets.
16         vector<int> a, b;
17
18         // Process numbers in groups of 4 from largest to smallest,
19         // assigning to sets such that each group adds equal sum to both.
20         while (n > 3 && n > 0) {
21             a.push_back(n);
22             a.push_back(n-3);
23
24             b.push_back(n-1);
25             b.push_back(n-2);
26
27             n = n - 4;
28     }
```

C++

```

29
30     // Handle the remaining 3 numbers if n % 4 == 3 (balanced
31     assignment).
32     if (n == 3) {
33         a.push_back(3);
34         b.push_back(2);
35         b.push_back(1);
36     }
37
38     // Output size and elements
39     cout << a.size() << endl;
40     for (int num : a)
41         cout << num << " ";
42
43     cout << b.size() << endl;
44     for (int num : b)
45         cout << num << " ";
46 }
47 }

```

1.2.9. Bit Strings

[Question - Bit Strings](#) [Backup Link](#)

Intuitive Explanation :

Each of the n positions can be either 0 or 1, so the answer is simply 2^n . We compute the power iteratively while taking remainders modulo $1e9+7$ to avoid overflow.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      const long long MOD = 1e9 + 7;
6      long long n;
7      cin >> n;
8
9      long long answer = 1;
10     for (long long i = 0; i < n; ++i) {
11         answer = (answer * 2) % MOD;
12     }
13
14     cout << answer << "\n";
15     return 0;
16 }
```

C++

1.2.10. Trailing Zeros

[Question - Trailing Zeros](#) [Backup Link](#)

Explanation :

The problem asks for the number of trailing zeros in n factorial. Zeros come from factor pairs of 2s and 5s. There will be excess 2s. Therefore the number of 5s alone determine the number of zeros.

Each multiple of 5 (5, 10, 15, 20, 25...) contributes one 5. Each multiple of 25 (25, 50, 75, 100, 125...) contributes an additional 5. Each multiple of 125 contributes another 5, and so on. The code loops through powers of 5 and counts the total number of the factor 5 present in n factorial.

Eg : $n = 27$

- $\lfloor \frac{27}{5} \rfloor = 5$ (5s from 5, 10, 15, 20, 25).
- $\lfloor \frac{27}{25} \rfloor = 1$ (extra 5 from 25).
- $\lfloor \frac{27}{125} \rfloor = 0$ (stop).
- Total: $5 + 1 + 0 = 6$ zeros.

Note : want to add reference as to what the meaning of the floor function is...

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n, count = 0;
6      cin >> n; // Read input number n
7
8      // Count factors of 5 in n! by summing n/5 + n/25 + n/125 + ...
9      //
10     for (int i = 5; n / i >= 1; i *= 5) {
11         count += n / i; // Add number of multiples of i (powers of 5)
12     }
13
14     cout << count << endl; // Output the number of trailing zeros
15     return 0;
16 }
```

C++

1.2.11. Coin Piles

[Question - Trailing Zeros](#) [Backup Link](#)

Explanation :

There are two key observations in this question :-

The first key observation is that each time you remove 2 coins from pile A and 1 coin from pile B or 1 coin from pile A and 2 coins from pile B, the total number of coins in both the towers always gets reduced by 3 so to empty both piles, the sum of coins in the two piles must be divisible by 3.

The second key observation is that the number of coins in one pile cannot exceed twice the number of coins in the other pile. Because in that case you cannot empty the bigger pile even if you remove 2 coins from the bigger pile for each time you remove 1 coin from the smaller pile.

Using this we check if the above two conditions are met and accordingly output the result.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int t;
6      cin >> t;
7
8      while (t--) {
9          int a, b;
10         cin >> a >> b;
11
12         bool cond_1 = (a + b) % 3 == 0;
13         bool cond_2 = max(a, b) <= 2 * min(a, b);
14
15         if(cond_1 && cond_2) {
16             cout<<"YES"<<endl;
17         }
18         else {
19             cout<<"NO";
20         }
21     }
22
23     return 0;
24 }
25
```

C++

1.2.12. Palindrome Reorder

[Question - Palindrome Reorder](#) [Backup Link](#)

Intuitive Explanation :

We start by counting the frequency of each letter. If there is an even number of characters, odd frequencies are not allowed, since a palindrome would be impossible. If there is an odd number of characters, only one letter with an odd frequency is allowed, as it can be placed in the center (for example, “aba”). Otherwise, the program builds the palindrome by placing characters symmetrically from both ends and putting any leftover odd-frequency character in the middle. The final constructed string is then printed, completing the rearrangement.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      string s;
6      cin >> s; // Read input string
7
8      int arr[26] = {}; // frequency array for letters A-Z (all initialized to 0)
9
10     // Count how many times each character appears
11     for (char c : s)
12         arr[c - 'A']++;
13
14     // Count how many characters have odd frequencies
15     int oddCount = 0;
16     for (int i = 0; i < 26; i++)
17         if (arr[i] % 2 != 0)
18             oddCount++;
19
20     // Palindrome rule:
21     // - If string length is even → no odd frequencies allowed
22     // - If string length is odd → exactly one odd frequency allowed
23     if (oddCount > 1 && s.size() % 2 == 1) {
24         cout << "NO SOLUTION"; // too many odd-count letters
25         return 0;
26     }
27     else if (oddCount > 0 && s.size() % 2 == 0) {
28         cout << "NO SOLUTION"; // even-length string with odd-count letters
29         return 0;
30     }
```

C++

```

31     else {
32         // Container to build the palindrome result
33         vector<char> str(s.length());
34         int left = 0, right = s.length() - 1;
35
36         // Fill symmetric pairs from both sides
37         for (int i = 0; i < 26; i++) {
38             while (arr[i] >= 2) {
39                 str[left++] = (char)('A' + i); // place letter on left side
40                 str[right--] = (char)('A' + i); // place same letter on
41                 // right
42                 arr[i] -= 2; // remove two occurrences
43             }
44         }
45
46         // If one odd-count character remains, put it in the middle
47         for (int i = 0; i < 26; i++)
48             if (arr[i] == 1)
49                 str[left] = (char)('A' + i);
50
51         // Convert vector<char> back to a string
52         s = string(str.begin(), str.end());
53
54         // Print the final palindrome
55         cout << s << endl;
56         return 0;
57     }

```

1.2.13. Gray Code

[Question - Gray Code](#) [Backup Link](#)

Hint:

Trying listing out the solution for $n = 1$, then for $n = 2$ and $n = 3$. Try to see if there is any pattern from the previous smaller sequences to the larger ones. You might even find a pattern just by looking at any one value of n .

1.2.13.1. Solution 1

The first pattern you may have spotted when you attempt to solve the question was the way the sequences of a longer Gray code build up on the sequence of smaller Gray code.

Take the Gray code for $n = 2$:

00
01
11
10

If you now look at the Gray code for $n = 3$, you'll notice that the gray code for $n = 2$ appears in the list:

000
001
011
010
110
111
101
100

The first 4 strings in the Gray code for $n = 3$ is just the Gray code for $n = 2$ (Shown in red) with 0 prepended to it. The last 4 string in the Gray code for $n = 3$ is just the gray code for $n = 2$ but backwards (Shown in blue). This pattern applies to all gray codes.

Here's the code for this approach:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios_base::sync_with_stdio(0);
6      cin.tie(0);
7      cout.tie(0);
8
9      int n;
10     cin >> n;
11
12     vector<string> gray;
```

C++

```

13     gray.push_back("");
14
15     for (int i = 0; i < n; i++) {
16         vector<string> next;
17
18         // Prefix "0" to all existing strings
19         for (int j = 0; j < gray.size(); j++)
20             next.push_back("0" + gray[j]);
21
22         // Prefix "1" to all existing strings in reverse order
23         for (int j = gray.size() - 1; j >= 0; j--)
24             next.push_back("1" + gray[j]);
25
26         gray = next;
27     }
28
29     for(int i = 0; i < gray.size(); i++)
30         cout << gray[i] << '\n';
31 }

```

From the code, we start by saying `gray = {""}`. Then to generate the next gray code which we store in `next`, we prepend a 0 to every string in `gray` and store that in `next` and then we prepend a 1 to every element in `gray` while going backwards and store that in `next`. Finally we update `gray` to `next`.

This process is repeated until you get the n th Gray Code which you then output.

The time complexity of this solution is $O(n \cdot 2^n)$ and the space complexity is $O(n)$. While this is pretty good and will pass all the test cases within the time limit, we can do a bit better.

1.2.13.2. Solution 2

Let's look again at the Gray code for $n = 3$ and highlight where the bit flips occur going from 000 to the end:

```

210
000
001
011
010
110
111
101
100

```

The numbers in gray at the top, represent the index of each bit. If we list the index of which bit was flipped from one string to the other, we get: $\{0, 1, 0, 2, 0, 1, 0\}$. In this list, you can see that we flipped the 0th bit every 2nd bit flip, the 1st bit every 4th bit flip, and if we were to look at Gray code of $n = 4$, we would also notice that the 2th bit was flipped every 8th time.

This sequence has a pattern that can be expressed with the following expression:

$$t_n = \log_2(\text{lsb}(n)), \text{ Where } t_n \text{ is the } n^{\text{th}} \text{ term.}$$

See for what LSB means.

We can use this formula to calculate the position of which bit we need to flip to generate the next string. The advantage of this method is that we don't need to compute the Gray codes for smaller values and we can just directly output the answer for the given n value.

Here's the code for this approach:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios_base::sync_with_stdio(0);
6      cin.tie(0);
7      cout.tie(0);
8
9      int n;
10     cin >> n;
11     string s(n, '0');// Start with all zeros
12
13     cout << s << endl;
14     for (int i = 1; i < (1 << n); i++) {
15         // Find position of lowest set bit and flip the corresponding bit
16         int pos = n - 1 - __builtin_ctz(i & -i);
17         s[pos] = (s[pos] == '0' ? '1' : '0');// Flip the bit.
18         cout << s << '\n';
19     }
20     return 0;
21 }
```

$i \& -i$ computes $\text{LSB}(n)$ and $\text{__builtin_ctz}()$ computes the number of trailing zeros which is the same as $\log_2()$ for a power of 2 which the $\text{LSB}(n)$ is. Finally we must subtract this from $n - 1$ to convert it to the correct index in the string because strings are indexed left to right but our formula indexes the bit's from right to left.

The time complexity of this code is $O(2^n)$ which is a bit faster than the first approach. For the last testcase of the problem on the website, the first code takes 0.03s whereas this one runs in 0.01s.

1.2.14. Tower of Hanoi

[Question - Tower of Hanoi](#) [Backup Link](#)

Hint:

Think about a recurrence relation that relates the solution of n and $n - 1$. Then you can write the code as either recursion(easy) or as a loop(hard).

Solution:

The recurrence relation is as follows: to move a stack of n disks from a starting pillar to the ending pillar, you must first move $n-1$ disks from the starting pillar to the middle pillar, then move the n^{th} disk from start to end, and then finally move the $n-1$ disks from the middle pillar to the end pillar.

Here's the simple recursion code which solves the question.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios_base::sync_with_stdio(0);
6      cin.tie(0);
7      cout.tie(0);
8
9      int n;
10     cin >> n;
11
12     vector<pair<int, int>> moves;
13
14     // This can also be a normal function instead of a lambda expression.
15     // Make sure that the moves vector is a global for this to work.
16     function<void (int, int, int, int)> solve = [&] (int n, int start, int end,
17     int middle){
18         if (n == 0) return;
19         solve(n - 1, start, middle, end);
20         moves.push_back({start, end});
21         solve(n - 1, middle, end, start);
22     };
23
24     //output:
25     solve(n, 1, 3, 2);
26     cout << moves.size() << endl;
27     for(int i = 0; i < moves.size(); i++)
28         cout << moves[i].first << " " << moves[i].second << endl;
29     return 0;
```

C++

29 }

As an extra challenge to the reader, try writing the loop solution.

1.2.15. Creating Strings

[Question - Trailing Zeros](#) [Backup Link](#)

Explanation :

In c++ there is a very useful function called 'next_permutation' which helps us tackle this exact question. This function can be used to generate the next lexicographical sequence for a string or a vector.

It returns false when no other greater permutations exists, otherwise it rearranges the string or the vector.

Note : Apurva add your explanation to the next permutation question...

Note : explain meaning of lexicographical down below

Code :

```
1
2  #include <bits/stdc++.h>
3  using namespace std;
4
5  int main() {
6      string s;
7      cin >> s;
8
9      //sort the string to get the lowest possible lexicographical sequence
10     sort(s.begin(), s.end());
11
12     vector<string> v;
13
14     do {
15         v.push_back(s);
16     } while (next_permutation(s.begin(), s.end()));
17     // returns false if no other permutation exists
18     // otherwise it rearranges the string
19
20     cout<<v.size()<<"\n";
21     for (int i = 0; i < v.size(); i++) {
22         cout << v[i] << endl;
23     }
24 }
```

C++

1.2.16. Apple Division

[Question - Apple Division](#) [Backup Link](#)

Intuitive Explanation :

The problem asks you to split the apples into two groups so that their total weights differ as little as possible. By checking every subset with bitmasks, you compute the sum of one group and compare it with the other using `abs(total - 2*sum)`. The smallest such difference across all subsets is the optimal answer.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      vector<int> v(n);
9      long long total = 0;
10
11     // Read weights and compute total sum.
12     for (int i = 0; i < n; i++) {
13         cin >> v[i];
14         total += v[i];
15     }
16
17     // ans = minimum possible difference between the two groups.
18     long long ans = total;
19
20     // Enumerate all subsets using bitmasks from 0 to (2^n - 1).
21     // Each mask chooses some apples for the first group;
22     // the rest naturally fall into the second group.
23     for (int mask = 0; mask < (1 << n); mask++) {
24
25         long long sum = 0;
26
27         // Compute sum of elements included in this subset.
28         for (int i = 0; i < n; i++) {
29             if (mask & (1 << i)) {
30                 sum += v[i];
31             }
32         }
33     }
```

C++

```
34         // If subset sum is S, the other group's sum is total - S.  
35         // Difference is |(total - S) - S| = |total - 2S|.   
36         ans = min(ans, labs(total - 2 * sum));  
37     }  
38  
39     cout << ans;  
40 }
```

1.2.17. Chessboard and Queens

[Question - Chessboard and Queens](#) [Backup Link](#)

Intuitive Explanation :

This solution uses backtracking² to count all valid ways to place 8 queens on an 8×8 board with blocked cells. We place exactly one queen per row, moving row by row. For each cell, we first skip blocked positions marked by *. Three boolean arrays track conflicts: columns, main diagonals ($\text{row} + \text{col}$), and anti-diagonals ($\text{row} - \text{col} + 7$). If a position is safe, we mark these arrays and recurse to the next row. When all 8 rows are filled, one valid arrangement is found and counted. After recursion, we backtrack by unmarking the position to explore other possibilities.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int N = 8;
5
6  vector<string> board(N);
7
8  // Tracking used columns and diagonals
9  bool usedColumn[N];
10 bool usedMainDiag[2 * N - 1]; // row + col
11 bool usedAntiDiag[2 * N - 1]; // row - col + (N - 1)
12
13 int totalWays = 0;
14
15 void placeQueen(int row) {
16     // All rows filled → one valid arrangement
17     if (row == N) {
18         totalWays++;
19         return;
20     }
21
22     for (int col = 0; col < N; col++) {
23         if (board[row][col] == '*') continue;
24
25         int mainDiag = row + col;
26         int antiDiag = row - col + (N - 1);
27
28         if (usedColumn[col] || usedMainDiag[mainDiag] ||
            usedAntiDiag[antiDiag])
```

C++

²backtracking was explained here

```

29         continue;
30
31         // Place queen
32         usedColumn[col] = usedMainDiag[mainDiag] = usedAntiDiag[antiDiag] =
            true;
33
34         placeQueen(row + 1);
35
36         // Backtrack
37         usedColumn[col] = usedMainDiag[mainDiag] = usedAntiDiag[antiDiag] =
            false;
38     }
39 }
40
41 int main() {
42     for (int i = 0; i < N; i++) {
43         cin >> board[i];
44     }
45
46     placeQueen(0);
47     cout << totalWays << "\n";
48     return 0;
49 }

```

1.2.18. Raab Game I

[Question - Raab Game I](#) [Backup Link](#)

Intuitive Explanation :

For each test case, we first check feasibility: if $a + b > n$, the required outcomes exceed the number of elements, then it is impossible. If exactly one of a or b is zero, the conditions become invalid.

When valid, we begin by printing numbers from 1 to n , representing the moves of the first player. The second line, representing the corresponding moves of the second player, is constructed carefully to control pairwise comparisons. The first b elements are taken from the range $a + 1$ to $a + b$, ensuring they are strictly larger than the next block and hence guarantee b wins. Next, the smallest a elements, 1 to a , are placed, ensuring wins for the first player. The remaining elements are appended in increasing order, resulting in draws for the remaining positions. This construction satisfies all constraints while maintaining valid permutations.

Code :

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int testCases;
6      cin >> testCases;
7
8      while (testCases--) {
9          int n, a, b;
10         cin >> n >> a >> b;
11
12         // Invalid if required elements exceed n
13         if (a + b > n) {
14             cout << "NO\n";
15             continue;
16         }
17
18         // Invalid if exactly one of a or b is zero
19         if ((a == 0 || b == 0) && a + b != 0) {
20             cout << "NO\n";
21             continue;
22         }
23
24         cout << "YES\n";
25
26         // 1 to n
```

C++

```

27     for (int i = 1; i <= n; i++) {
28         cout << i << " ";
29     }
30     cout << "\n";
31
32     // First b elements after a
33     for (int i = 1; i <= b; i++) {
34         cout << a + i << " ";
35     }
36
37     // Next a smallest elements
38     for (int i = 1; i <= a; i++) {
39         cout << i << " ";
40     }
41
42     // Remaining elements
43     for (int i = a + b + 1; i <= n; i++) {
44         cout << i << " ";
45     }
46     cout << "\n";
47 }
48
49 return 0;
50 }

```

1.2.19. Mex Grid Construction

[Question - Mex Grid Construction](#) [Backup Link](#)

Intuitive Explanation :

We fill the grid row by row, left to right. For each cell, we collect all values already placed to its left in the same row and above it in the same column. The cell is assigned the **mex** (smallest non-negative integer not present in those values). Since all needed cells are already filled, this greedy process always works.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      vector<vector<int>> grid(n, vector<int>(n, 0));
9
10     for (int i = 0; i < n; i++) {
11         for (int j = 0; j < n; j++) {
12             // Track used numbers
13             vector<bool> used(2 * n, false);
14
15             // Left in the same row
16             for (int k = 0; k < j; k++) {
17                 used[grid[i][k]] = true;
18             }
19
20             // Above in the same column
21             for (int k = 0; k < i; k++) {
22                 used[grid[k][j]] = true;
23             }
24
25             // Find mex
26             int mex = 0;
27             while (used[mex]) mex++;
28
29             grid[i][j] = mex;
30         }
31     }
32
33     // Output the grid
```

C++

```
34     for (int i = 0; i < n; i++) {
35         for (int j = 0; j < n; j++) {
36             cout << grid[i][j] << (j + 1 < n ? ' ' : '\n');
37         }
38     }
39
40     return 0;
41 }
```


1.2.20. Knight Moves Grid

[Question - Knight Moves Grid](#)

[Backup Link](#)

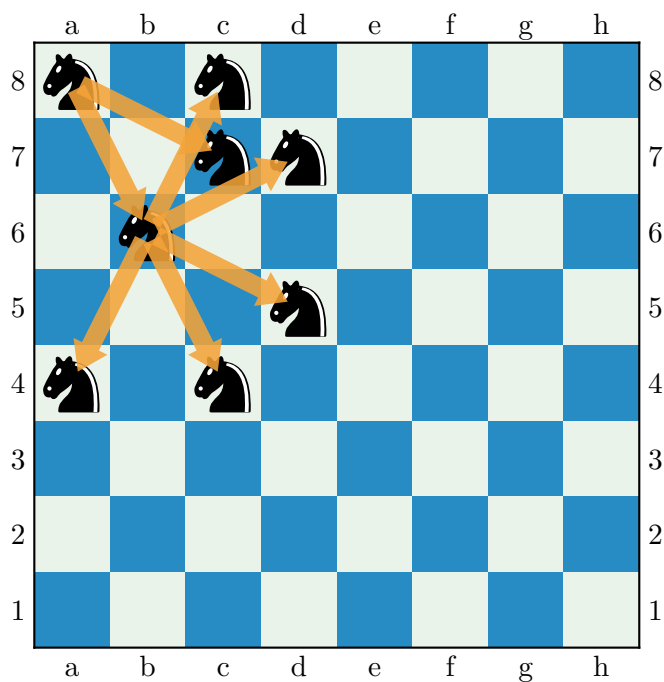
Explanation :

The program calculates the minimum number of knight moves needed to reach every square on an $n \times n$ chessboard starting from the top-left corner.

It keeps a grid where each cell stores how many moves are required to reach it, marking unreachable cells as -1 . The knight starts at position $(0, 0)$ with zero moves taken. From each position, all eight legal knight moves are tried. Whenever a new square is reached for the first time, its move count is recorded as one more than the current square. Each newly reached position is added so its moves can be explored later.

This process continues until all reachable squares have been processed. Finally, the grid of minimum move counts is printed.

A visual understanding of the algorithm can be found in the image below:



You move from the knight to all unvisited grid. This approach guarantess the shortest path to the target.

Code :

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
```

C++

```

4
5  int main() {
6      int n;
7      cin >> n;
8
9      // dist[x][y] will store the minimum number of moves needed to reach cell
      (x, y)
10     // a value of -1 means that cell has not been reached yet
11     vector<vector<int>> dist(n, vector<int>(n, -1));
12
13     // This queue stores board positions that still need to be explored
14     queue<pair<int, int>> q;
15
16     // These arrays describe how a knight moves on a chessboard
17     // Each (dx[k], dy[k]) pair represents one possible knight move
18     vector<int> dx = {-2, -1, 2, 1, 2, 1, -1, -2};
19     vector<int> dy = {-1, -2, 1, 2, -1, -2, 2, 1};
20
21     // This function checks whether a position is inside the board
22     // and whether it has not been visited before
23     auto isValid = [&](int x, int y) {
24         return x >= 0 && y >= 0 && x < n && y < n && dist[x][y] == -1;
25     };
26
27     // We begin from the top-left cell (0, 0)
28     // Reaching the starting cell takes 0 moves
29     dist[0][0] = 0;
30     q.push({0, 0});
31
32     // As long as there are positions left to explore, keep processing them
33     while (!q.empty()) {
34         // Take the oldest unexplored position from the queue
35         auto [x, y] = q.front();
36         q.pop();
37
38         // Try moving the knight in all 8 possible ways from this position
39         for (int k = 0; k < 8; k++) {
40             int nx = x + dx[k];
41             int ny = y + dy[k];
42
43             // If the new position is valid and not visited yet
44             if (isValid(nx, ny)) {
45                 // The distance to this cell is one more than the current
46                 cell
47                 dist[nx][ny] = dist[x][y] + 1;

```

```

47
48         // Add the new position to the queue to explore later
49         q.push({nx, ny});
50     }
51 }
52 }
53
54 // Print the minimum number of moves needed to reach each cell
55 for (int i = 0; i < n; i++) {
56     for (int j = 0; j < n; j++) {
57         cout << dist[i][j] << " ";
58     }
59     cout << endl;
60 }
61
62 return 0;
63 }
64 }

```

1.2.21. Grid Coloring I

[Question - Grid Coloring I](#) [Backup Link](#)

Intuitive Explanation :

For each cell maintain an array of 4 booleans tracking which colors (A-D) are available. Now there are three conditions:

1. Mark the current cell's original color as invalid.
2. Mark the color of the cell above it in the `ans` grid as invalid.
3. Mark the color of the cell to the left of it in the `ans` grid as invalid.

Now we can greedily assign the first available color to the `ans` grid. As there are 4 colors and only 3 conditions, we can always find a valid color.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n, m;
6      cin >> n >> m;
7
8      vector<string> v(n);          // Input grid
9      char ans[n][m];              // Output grid with adjusted characters
10
11     // Read the input grid
12     for (int i = 0; i < n; i++)
13         cin >> v[i];
14
15     // Construct the output grid
16     for (int i = 0; i < n; i++) {
17         for (int j = 0; j < m; j++) {
18
19             // Tracks whether letters A, B, C, D are allowed at this cell
20             bool isValid[4] = {true, true, true, true};
21
22             // Block the original character in this position
23             isValid[v[i][j] - 'A'] = false;
24
25             // Block the character above (if exists)
26             if (i > 0)
27                 isValid[ans[i - 1][j] - 'A'] = false;
28
29             // Block the character to the left (if exists)
```

C++

```

30         if (j > 0)
31             isValid[ans[i][j - 1] - 'A'] = false;
32
33         // Choose the first valid character using your four ifs
34         if (isValid[0])
35             ans[i][j] = 'A';
36         else if (isValid[1])
37             ans[i][j] = 'B';
38         else if (isValid[2])
39             ans[i][j] = 'C';
40         else if (isValid[3])
41             ans[i][j] = 'D';
42     }
43 }
44
45 // Print the final grid
46 for (int i = 0; i < n; i++) {
47     for (int j = 0; j < m; j++)
48         cout << ans[i][j];
49     cout << "\n";
50 }
51 }

```

1.2.22. Digit Queries

[Question - Digit Queries](#) [Backup Link](#)

Intuitive Explanation :

The trick is to notice that numbers form blocks by digit-length: 1–9 (1-digit), 10–99 (2-digit), 100–999 (3-digit), and so on. Each block contributes a predictable number of digits, so the code keeps subtracting whole blocks until the target digit falls inside one specific block. Once the block is located, it directly computes which exact number contains the digit, converts that number to a string, and extracts the correct character. This avoids generating any sequence and keeps the solution fast even for huge positions.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4
5  int main() {
6      ios::sync_with_stdio(false);
7      cin.tie(nullptr);
8
9      int q;
10     cin >> q;
11
12     while (q--) {
13         ll n;
14         cin >> n;
15
16         ll count = 9;    // how many numbers exist with current digit-length
17         ll len = 1;      // current digit-length
18         ll start = 1;    // first number of current digit-length
19
20         // skip full digit-blocks (e.g., 1–9, then 10–99, then 100–999...)
21         while (n > count * len) {
22             n -= count * len; // remove whole block worth of digits
23             start *= 10;      // move to next block's starting number
24             count *= 10;      // next block has 10× more numbers
25             len++;           // numbers now have one more digit
26         }
27
28         // find the exact number containing the nth digit
29         ll num = start + (n - 1) / len;
30
31         // pick the specific digit inside that number
```

C++

```
32     string s = to_string(num);
33     cout << s[(n - 1) % len] << "\n";
34 }
35 }
```

1.2.23. String Reorder

[Question - String Reorder](#) [Backup Link](#)

Intuitive Explanation :

The program rearranges a string so that no two adjacent characters are the same while keeping the result lexicographically smallest. It maintains a frequency array of remaining letters and builds the answer one character at a time.

At each step, it checks whether a valid rearrangement is still possible by ensuring no character occurs more than half of the remaining length.

If a character is too frequent, it is forced to be chosen immediately to avoid failure. Otherwise, the smallest lexicographically valid character different from the previous one is selected.

If at any point no valid choice exists, the program outputs -1

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // Returns the lexicographically smallest valid next character index
5  // freq[] stores remaining frequencies of letters A-Z
6  // prev = -1 means no previous character (first position)
7  // prev >= 0 means index of previous character used
8  int minLexPossible(int freq[], int prev) {
9      int maxLetter = 0, sum = 0;          // maxLetter = highest frequency, sum
      = total remaining letters
10     int minLetter = -1, maxIndex = 0; // minLetter = smallest valid choice,
      maxIndex = most frequent letter
11
12     // Find the smallest lexicographically valid letter different from prev
13     for (int i = 0; i < 26; i++) {
14         if (freq[i] > 0 && i != prev) {
15             minLetter = i;
16             break;
17         }
18     }
19
20     // Compute total remaining letters and the letter with maximum frequency
21     for (int i = 0; i < 26; i++) {
22         sum += freq[i];
23         if (freq[i] > maxLetter) {
24             maxLetter = freq[i];
25             maxIndex = i;
```

C++


```

26     }
27 }
28
29 // If any letter appears too often, rearrangement is impossible
30 if (maxLetter * 2 > sum + 1) return -1;
31
32 // If the most frequent letter must be placed now to avoid failure, force
  it
33 if (maxLetter * 2 > sum) return maxIndex;
34
35 // Otherwise, choose the smallest lexicographically valid letter
36 return minLetter;
37 }
38
39 int main() {
40     string s, ans = ""; // s = input string, ans = constructed result
41     cin >> s;
42
43     int freq[26] = {0}; // Frequency array for letters A-Z
44     for (char c : s) freq[c - 'A']++;
45
46     // Choose the first character (no previous restriction)
47     int idx = minLexPossible(freq, -1);
48     if (idx == -1) {
49         cout << "-1"; // Impossible to form valid string
50         return 0;
51     }
52
53     ans += char(idx + 'A'); // Append chosen character
54     freq[idx]--;           // Decrease its frequency
55
56     // Build the rest of the string character by character
57     for (int i = 1; i < s.size(); i++) {
58         // Previous character index is ans[i - 1] - 'A'
59         idx = minLexPossible(freq, ans[i - 1] - 'A');
60         if (idx == -1) {
61             cout << "-1"; // No valid continuation
62             return 0;
63         }
64         ans += char(idx + 'A'); // Append next character
65         freq[idx]--;           // Update frequency
66     }
67
68     cout << ans; // Output the lexicographically smallest valid arrangement
69 }

```


1.2.24. Grid Path Description

[Question - Grid Path Description](#) [Backup Link](#)

Intuitive Explanation :

The program counts how many valid ways exist to move through a 7×7 grid using exactly 48 moves. You start at the top-left cell and must end at the bottom-left cell without visiting any cell twice. Each move must follow the given string, where ? allows any direction and other characters force a specific move. The code tries all allowed moves step by step while marking visited cells.

Paths that get trapped or split the grid into unreachable regions are stopped early. Every complete path that reaches the destination at exactly 48 steps is counted.

Code :

```
1  using namespace std;
2
3  string path;
4  bool visited[7][7];          // Marks cells already used in the current path
5  int ans = 0;
6
7  const int dx[] = {1, -1, 0, 0}; // Change in row for each move
8  const int dy[] = {0, 0, 1, -1}; // Change in column for each move
9  const char dir[] = {'D', 'U', 'R', 'L'}; // Corresponding move letters
10
11 bool inside(int x, int y) {
12     return x >= 0 && x < 7 && y >= 0 && y < 7; // Checks grid boundaries
13 }
14
15 bool is_blocked(int x, int y) {
16     if (!inside(x, y) || visited[x][y]) return true; // Outside grid or
17     // already used
18     return false;
19 }
20 void dfs(int x, int y, int step) {
21     // If we reached the target cell
22     if (x == 6 && y == 0) {
23         if (step == 48) ans++; // Count only if all moves were used
24         return;
25     }
26
27     // Stop early if movement is forced into a dead split
28     if ((is_blocked(x + 1, y) && is_blocked(x - 1, y) &&
29         !is_blocked(x, y + 1) && !is_blocked(x, y - 1)) ||
```

C++

```

30         (!is_blocked(x + 1, y) && !is_blocked(x - 1, y) &&
31         is_blocked(x, y + 1) && is_blocked(x, y - 1)))
32         return;
33
34     visited[x][y] = true;    // Mark this cell as used
35
36     for (int d = 0; d < 4; ++d) {
37         int nx = x + dx[d];
38         int ny = y + dy[d];
39
40         // Enforce the given path character if it is not '?'
41         if (path[step] != '?' && path[step] != dir[d]) continue;
42
43         // Move only to valid and unused cells
44         if (inside(nx, ny) && !visited[nx][ny])
45             dfs(nx, ny, step + 1);
46     }
47
48     visited[x][y] = false; // Undo the move before trying other
    possibilities
49 }
50
51 int main() {
52     cin >> path;
53     dfs(0, 0, 0);           // Start from top-left with zero moves taken
54     cout << ans << endl;
55     return 0;
56 }

```

